

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I Sowmiya. D (192511063) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sowmiya. D

Reverse Engineering

1. Machine Learning Techniques for Reverse Engineering

Description:

Machine Learning can be used to classify software components, identify programming patterns, detect anomalies, and predict the functionality of unknown code. Students study supervised, unsupervised, and semi-supervised learning approaches used in reverse engineering.

Student Activity:

- Collect a dataset of source-code samples.
- Extract code features.
- Apply classification or clustering algorithms.
- Analyze similarities among programs.
- Compare the performance of different ML algorithms.

Expected Outcome:

Students will evaluate the suitability of different ML algorithms for software reverse-engineering tasks.

2. AI for Source Code Analysis and Program Understanding

Description:

AI can analyze large source-code repositories and automatically identify functions, classes, variables, dependencies, and programming patterns. Natural Language Processing and Large Language Models can also convert complex code into human-readable explanations.

Student Activity:

- Select a source-code repository.
- Analyze the code using AI-assisted tools.
- Identify important functions and dependencies.
- Generate natural-language explanations.
- Verify the AI-generated explanations against the actual code.

Expected Outcome:

Students assess the effectiveness and limitations of AI in understanding unfamiliar source code.

3. AI-Based Malware Reverse Engineering and Analysis

Description:

AI can assist security researchers in analyzing malware behavior by identifying suspicious code patterns, API calls, strings, functions, and execution behaviors. Students can study the conceptual workflow of AI-assisted malware analysis using **safe, non-executable datasets or publicly available research samples**.

Student Activity:

- Study malware-analysis datasets.
- Identify relevant static features.
- Apply ML classification techniques.
- Analyze suspicious patterns.
- Evaluate classification accuracy.

Expected Outcome:

Students understand and evaluate how AI can support automated malware analysis.

4. Reverse Engineering of APIs Using AI

Description:

APIs may need to be understood when documentation is incomplete or unavailable. AI can analyze API requests, responses, source code, and observed behavior to infer endpoints, parameters, data formats, and relationships.

Student Activity:

- Select a documented/open API or controlled test API.
- Analyze its request-response structure.
- Use AI to infer API functionality.
- Generate documentation.
- Compare generated documentation with the actual API specification.

Expected Outcome:

Students assess the usefulness of AI for API understanding and documentation recovery.

1. Machine Learning Techniques for Reverse Engineering

- Machine Learning (ML) can significantly assist reverse engineering by automatically identifying patterns and relationships within source code and software components. The main approaches include supervised, unsupervised, and semi-supervised learning.
 - In a supervised learning approach, a dataset containing labeled source-code samples is collected. Each program can be labeled according to its programming language, functionality, algorithm type, or software category. Features such as keywords, operators, function names, control-flow structures, code length, cyclomatic complexity, and API calls can be extracted from the source code. Classification algorithms such as Decision Trees, Random Forest, Support Vector Machines, and Neural Networks can then be trained to classify unknown programs.
 - Unsupervised learning can be used when labeled data is unavailable. Algorithms such as K-Means and hierarchical clustering can group programs based on similarities in their source-code features. This can help identify related programs, duplicated code, or previously unknown software categories.
 - Semi-supervised learning combines a small amount of labeled data with a large amount of unlabeled data. This is useful in reverse engineering because manually labeling large software datasets can be expensive and time-consuming.
 - The student activity involves collecting source-code datasets, extracting relevant features, applying different ML algorithms, and comparing their performance using measures such as accuracy, precision, recall, and F1-score.
 - The expected outcome is that students understand how different ML techniques can support software classification, similarity analysis, anomaly detection, and program understanding, and can determine which algorithm is most suitable for a particular reverse-engineering task.
-

2. AI for Source Code Analysis and Program Understanding

- Artificial Intelligence can be used to analyze source code and help reverse engineers understand unfamiliar software systems. Modern AI systems, particularly Natural Language Processing (NLP) models and Large Language Models (LLMs), can process source code and generate human-readable explanations.
 - The first step is to select a source-code repository. The repository is analyzed to identify important components such as functions, classes, variables, modules, libraries, and dependencies. Static-analysis tools can be combined with AI to understand relationships between these components.
 - AI can also identify programming patterns and summarize individual functions. For example, an AI model may analyze a function and explain its inputs, processing steps, outputs, and possible dependencies. It can also describe how different modules interact with one another.
 - Students can select an open-source repository and use AI-assisted tools to analyze the source code. They can identify important functions and dependencies and generate natural-language explanations for selected portions of the program. These explanations must then be compared with the actual source code to determine whether the AI has interpreted the program correctly.
 - One important limitation is that AI-generated explanations may contain incorrect assumptions or hallucinations. Therefore, human verification is essential.
 - The expected outcome is that students learn how AI can accelerate source-code analysis while also understanding its limitations and the importance of validating AI-generated results against the actual implementation.
-

3. AI-Based Malware Reverse Engineering and Analysis

- AI can support malware reverse engineering by automatically identifying suspicious characteristics in software samples. Instead of manually examining every sample, security researchers can use ML techniques to classify samples and identify potentially malicious patterns.

- For safe educational analysis, students should work with non-executable datasets, static malware-analysis datasets, or controlled research samples. Relevant features may include imported libraries, API-call information, strings, file characteristics, instruction patterns, and other static indicators.
 - The collected data can be converted into numerical or categorical features that can be processed by ML algorithms. Classification algorithms such as Random Forest, Support Vector Machines, or Neural Networks can be trained to distinguish between benign and suspicious samples.
 - Anomaly detection can also be applied. A model can learn the characteristics of normal software and identify samples that significantly differ from the expected behavior. Clustering can group similar samples and potentially reveal relationships between different malware families in a research dataset.
 - Students can analyze a safe dataset, identify important static features, apply classification algorithms, and evaluate their performance using accuracy, precision, recall, and F1-score.
 - The expected outcome is that students understand how AI can automate parts of malware analysis, recognize suspicious patterns, and reduce the workload of security researchers. They should also understand that ML systems can produce false positives and false negatives and therefore require expert validation.
-

4. Reverse Engineering of APIs Using AI

- Application Programming Interfaces (APIs) allow software systems to communicate with one another. When API documentation is incomplete, reverse engineers may need to determine how the API works by studying its available source code or observing controlled request-response interactions.
- AI can assist this process by analyzing API requests, responses, parameters, data formats, and relationships between endpoints. For example, an AI system can examine several request-response pairs and identify common endpoint structures and parameter types.
- Students can select a documented or deliberately constructed test API and collect safe request-response examples. They can analyze HTTP methods such as GET, POST, PUT, and DELETE, along with parameters, headers, response codes, and JSON or XML data structures.
- AI can then be used to infer the likely purpose of different endpoints. For example, based on a request and response structure, it may determine that an endpoint is related to retrieving user information or creating a particular resource.
- The AI can also generate documentation describing the inferred endpoints, parameters, request formats, response formats, and expected functionality. Students then compare this generated documentation with the actual API specification.
- The expected outcome is that students learn how AI can support API reverse engineering, behavioral analysis, and documentation recovery. They also learn that AI-generated API descriptions must be validated because inferred functionality may be incomplete or incorrect.

Conclusion

- AI provides powerful assistance for reverse engineering by automating code classification, source-code understanding, malware analysis, and API analysis. Machine Learning is useful for identifying patterns and classifying software, while NLP and LLMs can translate complex code into understandable descriptions. However, AI results should always be verified by human experts because models can produce inaccurate interpretations, false positives, and incomplete conclusions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness